MODULE 27 · DOMAIN 4 - ASSEMBLING AND DEPLOYING APPLICATIONS

CI/CD and the prompt lifecycle

Testing every kind of change, deploying through bundles and deployment jobs, and versioning prompts with aliases.

SECTION 4 of 6 EXAM WEIGHT 22% MODULE 27 of 40

admin@sthithapragna.com

An alias is a bookmark, not a copy. **Move the bookmark to release. Move it back to roll back. The pages never change.**

CI/CD for a generative AI application

01
Change
Code, prompt, chunking, model, in Git.

02
Test
Unit tests, then evaluation with judges.

03
Gate
Pass thresholds, or stop.

04
Deploy
Bundles to staging, then approval.

05
Promote
Move the production alias. Monitor.

ON THE EXAM

The evaluation set from module 20 is the heart of the gate. Unit tests alone cannot tell you whether answers got worse.

Every kind of change, and how to test it

CHANGE	HOW TO TEST IT
Prompt wording	Evaluation set with judges
Tool or parser code	Unit tests, then evaluation
Chunking or embedding	Retrieval metrics, then a rebuilt index
Model swap	Evaluation, plus cost and latency
New guardrail	Log mode first, from module 17
New source documents	Index sync, then spot-check retrieval

ON THE EXAM

A prompt edit is a code change for testing purposes. It can break answers as badly as a bug.

Test components, or only the whole?

ONLY END TO END

One big evaluation

- Shows that something got worse
- Not where it got worse
- Slow to run on every change
- Failures are hard to diagnose

COMPONENTS AND THE WHOLE

Layered tests

- Unit tests for tools and parsers
- Retrieval metrics for the retriever
- End-to-end judges for answers
- A failure points to its component

THE TIEBREAKER

Test each component on its own, and the whole with an evaluation set. A failing layer tells you where to look.

Declarative Automation Bundles

- Formerly **Databricks Asset Bundles**. Renamed on 16 March 2026, still called DABs.
- A **databricks.yml** file declares jobs, pipelines, endpoints and apps as code.
- **Targets** such as dev, staging and prod deploy the same project to each environment.
- The bundle CLI validates, deploys and runs the project, typically from a CI system.
- The rename is non-breaking: commands and existing configuration are unchanged.

ON THE EXAM

Bundles make the whole application, not just the model, reproducible across environments.

MLflow 3 deployment jobs

- A job that runs **automatically when a new model version** is created in Unity Catalog.
- Typical steps: **evaluation, approval, deployment**.
- Approval can be a **human in the loop**, through the model's page in Unity Catalog.
- Every run is recorded in the model version's **activity log**.
- Currently in **Public Preview**.

> ON THE EXAM
>
> A deployment job turns the gate into something that cannot be skipped: no approval, no deployment.

The index is part of the release too

- New documents: the pipeline **triggers an index sync** after the data load.
- New chunking or embedding model: **build a new index**, then re-embed everything.
- Evaluate against the new index before switching the agent to it.
- Keep the index name in configuration, so switching is a config change.
- A triggered index that nobody syncs goes stale silently.

ON THE EXAM

Data changes need a sync step in the job. Chunking or model changes need a new index, evaluated before the switch.

Versioning prompts in Unity Catalog

ACTION	HOW
Register a version	mlflow.genai.register_prompt, name catalog.schema.prompt
Template variables	Double braces, such as {{question}}
Load a version	prompts:/name/3, a fixed version
Load by alias	prompts:/name@production, follows the bookmark
Move an alias	mlflow.genai.set_prompt_alias
Permissions	CREATE FUNCTION, EXECUTE and MANAGE on the schema

ON THE EXAM

Versions never change once registered. Aliases move. That is what makes release and rollback safe.

A hard-coded prompt, or a registry alias?

HARD-CODED PROMPT

Prompt inside the code

- Every edit means a redeploy
- No history of wording changes
- Rollback means another deploy
- Hard to test versions side by side

REGISTRY ALIAS

Prompt loaded by alias

- Move the alias, no redeploy
- Every version kept, with history
- Rollback is instant
- Versions easy to evaluate and compare

THE TIEBREAKER

Databricks recommends loading prompts by alias in deployed applications, rather than hard-coding versions.

From an edit to production, and back

01
Edit
Register a new version, with a commit message.

02
Evaluate
Run the evaluation set against it.

03
Stage
Move the staging alias. Test in staging.

04
Release
Move the production alias.

05
Roll back
If needed, move it back. Instantly.

ON THE EXAM

Every step is a bookmark move or an evaluation. No code is redeployed at any point.

How Harbourline releases changes

01
Git
Agent, tools and bundle YAML.

02
CI
Unit tests, retrieval metrics, 300-question gate.

03
Bundle
Deploy to staging, then production targets.

04
Model
Deployment job: evaluate, approve, deploy.

05
Prompts
Loaded by alias. Wording ships without redeploying.

ON THE EXAM

Compliance can see who changed what, when, and on what evidence. That record is part of the product.

> A team edits its agent's system prompt about once a week. The prompt is a string constant in the agent's code.
>
> Every edit needs a full redeploy of the agent, which takes an hour and needs sign-off.

What should the team change?

A Store the prompt text in the agent's ModelConfig YAML file

B Keep the constant, but automate the redeploy so it is faster

C Load the prompt by alias from the MLflow Prompt Registry

D Put the prompt into each user's first message of the chat

◆ PAUSE AND DECIDE

What should the team change?

A ModelConfig YAML

Better than code, but logged with the model: still a redeploy.

B Faster redeploy

Still a redeploy per wording change.

C Registry, by alias

Moving the alias ships a new version with no redeploy.

D In the user's message

Users could change or remove it. Unsafe.

ON THE EXAM

The prompt changes often, and every change forces a redeploy. Load it by alias, and releases become alias moves.

> Prompt version 7 was moved to the production alias this morning. Complaints about curt answers began within the hour.
>
> Version 6 worked well. The team needs to roll back immediately.

What is the fastest safe rollback?

A Point the production alias back at version 6 right away

B Delete version 7 from the registry, then redeploy the agent

C Edit version 7 in place to restore the older wording

D Register a new version 8 copied from version 6's text

◆ PAUSE AND DECIDE

What is the fastest safe rollback?

A Alias back to 6

One bookmark move. Every app follows at once.

B Delete 7, redeploy

Loses history, and a redeploy is not immediate.

C Edit 7 in place

Versions are immutable. It cannot be done.

D New version 8

Works, but slower, and adds a duplicate to the history.

ON THE EXAM

Roll back by moving the bookmark. Versions are never edited, and never need to be.

> A team wants every merge to the main branch to check automatically that answer quality has not dropped, before anything reaches production.
>
> It already has unit tests for its tool functions and a 250-question evaluation set.

What should the CI pipeline do?

A Have a reviewer try three questions by hand after each merge

B Rely on production monitoring to catch any drop in quality later

C Run the unit tests on the tool functions and skip evaluation

D Run the unit tests and the evaluation set with judges as a gate

◆ PAUSE AND DECIDE

What should the CI pipeline do?

A Three questions by hand

Not automatic, and far too small a sample.

B Production monitoring

Catches the drop after users have seen it.

C Unit tests only

Proves functions run, not that answers are good.

D Unit tests plus evaluation gate

Components and answer quality, checked before release.

ON THE EXAM

Automatic, answer quality, before production. Only an evaluation gate in CI meets all three.

> New policy documents land in the chunk table every night. The AI Search index uses triggered sync.
>
> Users report that answers never include documents added the previous night.

What should the engineer change?

A Move the index to a storage-optimised endpoint for faster indexing

B Add an index sync task after the nightly data load in the job

C Rebuild the whole index from scratch every night instead

D Raise num_results so that newer chunks are more likely

◆ PAUSE AND DECIDE

What should the engineer change?

A Storage-optimised

Faster indexing does nothing if no sync is triggered.

B Sync task after the load

Triggered sync needs a trigger. Put it in the pipeline.

C Rebuild nightly

Wasteful. An incremental sync picks up the new rows.

D More results

The new chunks are not in the index at all.

THE SIGNAL

Triggered sync, and new rows never appear. Nobody is triggering it: make the sync a step in the job.

What to carry forward

- CI/CD gate: **unit tests for components** plus an **evaluation set with judges**.
- **Declarative Automation Bundles**, formerly Asset Bundles, deploy the project to each target.
- MLflow 3 **deployment jobs**: evaluate, approve, deploy, on each new model version.
- Prompts: register versions, **load by alias**, move the alias to release or roll back.
- Triggered indexes need a sync step. Versions are immutable.

ON THE EXAM

Next: module 28 puts the application in front of users: Databricks Apps, Slack and Teams.